



EDUMIND: AI BEHAVIOR ANALYSIS SYSTEM

Prolog based Student Behavior Analyzer System

Submitted by :

**LAIBA LARAIB (SP24-BCS-053)
MUHAMMAD USAMA (SP24-BCS-078)**

Department of Computer Science, CUI Islamabad

Date of Submission : May 18, 2026

Table of Contents

Abstract	2
1. Introduction.....	3
2. Problem Statement	3
3. Proposed System	3
3.1 System Architecture	5
4. Advantages and Benefits.....	5
5. Scope of the Project	6
File Structure	7
6. Modules of the System.....	7
Module 1: Computer Vision Observation Module.....	7
Module 2: Feature Extraction & Observation Processing Module	8
Module 3: Fact Generation Module (Interface Layer).....	9
Module 4: Prolog Expert System (Core Intelligence Module)	9
Module 5: Backend Integration Module (Flask Server)	10
Module 6: Frontend Dashboard Module.....	11
6.Working of Prolog Expert System (Core Intelligence)	11
7. System Limitations	15
Future Enhancements.....	15
8. Tools and Technologies	16
8.1 Tools and Technologies Used	16
8.2 AI Techniques used :.....	16
9. GUI Mockups.....	17
10. Deployment of the System.....	23
11. Conclusion	23
12. References	23

Project Category: A- System

Abstract

EduMind is an Artificial Intelligence based intelligent expert system designed for real-time student behavior analysis. The system combines Computer Vision for observation extraction and a Prolog-based Expert System for intelligent reasoning and decision making.

The Computer Vision module uses OpenCV and MediaPipe to detect facial landmarks, eye movement, head posture, gaze direction, and hand activity. However, this module does not perform decision making; instead, it only generates raw behavioral observations.

These observations are then passed to a Prolog Knowledge Base, which acts as the core intelligence of the system. The expert system applies rule-based reasoning, backward chaining, and unification to analyze student behavior and classify engagement levels such as attentive, distracted, sleepy, or disengaged.

The system demonstrates how symbolic AI (Expert Systems) combined with Computer Vision observations can be used to build an explainable and intelligent decision-making system for educational environments.

1. Introduction

Maintaining student attention in classrooms and online learning environments is a major challenge for educators. Students often become distracted, tired, or disengaged during lectures, and manual monitoring becomes inefficient, especially in large classes.

To address this problem, EduMind is developed as an AI-based intelligent expert system that analyzes student behavior using a combination of Computer Vision and Prolog-based reasoning.

The system uses a webcam to capture real-time video of the student. OpenCV and MediaPipe are used only to extract visual observations such as eye closure, head orientation, gaze direction, and hand movement. These observations are not interpreted by the vision module itself.

Instead, all decision-making is handled by a Prolog Knowledge Base, which acts as the brain of the system. It uses logical rules and inference techniques to determine student behavior and engagement level.

The project helps demonstrate important Artificial Intelligence concepts including:

- Expert Systems
- Knowledge Representation
- Rule-Based Reasoning
- Backward Chaining
- Unification
- Computer Vision (as perception layer)
- Explainable AI

2. Problem Statement

Teachers and instructors face difficulty in continuously monitoring student attention and engagement during classroom lectures and online learning sessions. Students often become distracted, sleepy, disengaged, or use mobile phones without immediate detection. Existing systems lack intelligent reasoning capabilities and mostly rely on manual observation.

The problem addressed by this project is:

“How can Artificial Intelligence be used to automatically monitor and analyze student behavior in real time and intelligently determine attention and engagement levels?”

EduMind solves this problem using Computer Vision and Expert System technique of Prolog

3. Proposed System

EduMind is designed as a two-layer intelligent system consisting of a perception layer and a reasoning layer.

Scope document for EduMind

The first layer is the Computer Vision module, which captures live webcam frames and extracts raw observations using OpenCV and MediaPipe. This module identifies visual signals such as eye closure, head down posture, gaze direction, and hand movement near the face. However, it does not make any decisions.

The second layer is the Prolog-based Expert System, which is responsible for all reasoning and decision making. The observations generated by the vision module are converted into logical facts and passed into the Prolog Knowledge Base.

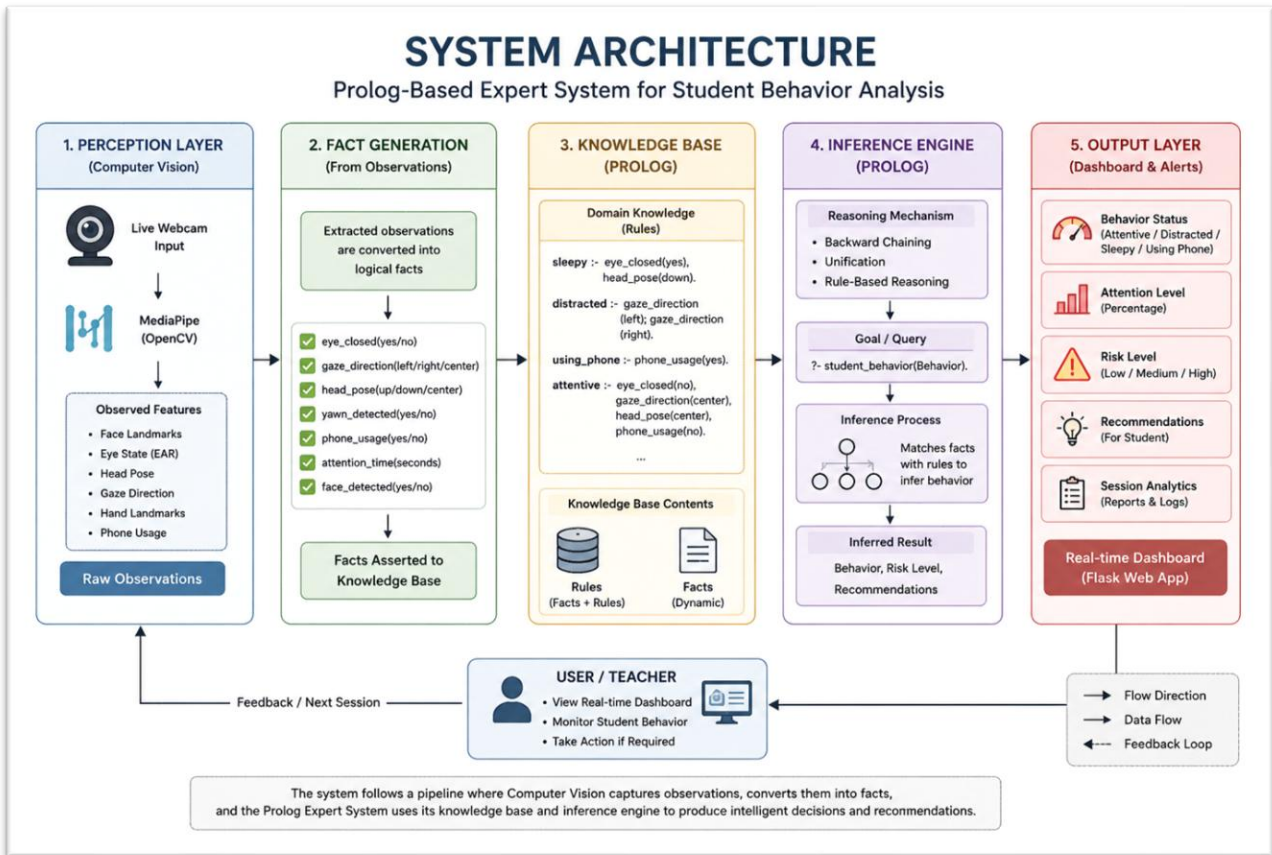
The Prolog system uses predefined rules to infer student behavior states such as:

- attentive_student
- distracted_student
- sleepy_student
- disengaged_student
- phone_usage_suspected

The system applies rule-based reasoning and backward chaining to derive conclusions from observed facts.

The final output is displayed through a Flask-based web dashboard showing attention level, behavior status, risk level, and recommendations

3.1 System Architecture



4. Advantages and Benefits

The EduMind system provides multiple advantages over traditional classroom monitoring methods:

1. Real-time student attention monitoring.
2. Intelligent AI-based behavior analysis.
3. Explainable AI through logical reasoning.
4. Reduced manual observation workload.
5. Detection of distraction and sleepiness.
6. Personalized recommendations.
7. Improved classroom monitoring efficiency.
8. Integration of Computer Vision and Expert Systems.

9. Educational implementation of AI concepts.
10. Modular and scalable architecture.
11. Reduced false detections through temporal validation.
12. Live analytics and monitoring dashboard.

5. Scope of the Project

- Real-time observation of student behavior using webcam
- Extraction of facial and hand landmarks using Computer Vision
- Generation of logical facts from visual observations
- Intelligent reasoning using Prolog Expert System
- Classification of student engagement levels
- Rule-based decision making using AI techniques
- Real-time web-based dashboard for monitoring
- Flask-based deployed application for system interaction

Out of Scope

- Multi-user classroom monitoring
- Emotion and speech recognition

File Structure

FILE STRUCTURE OF THE PROJECT		
Prolog-Based Expert System for Student Behavior Analysis		
FOLDER / FILE	DESCRIPTION	TYPE
Student_Behavior_Expert_System/	Root directory of the project	Directory
app/	Main application files (Flask backend & core logic)	Directory
app.py	Entry point of the Flask application	Python File
routes.py	Handles web routes and API endpoints	Python File
dashboard.py	Dashboard data processing and visualization logic	Python File
prolog_engine.py	Interface between Python and Prolog (pyswip)	Python File
utils.py	Helper functions and common utilities	Python File
cv_module/	Computer Vision module (Observation layer)	Directory
camera.py	Handles webcam feed and frame capture	Python File
feature_extractor.py	Extracts observations (EAR, head pose, gaze, phone, etc.)	Python File
landmark_detector.py	Face, eye and hand landmarks detection (MediaPipe)	Python File
prolog_kb/	Prolog knowledge base and rules	Directory
knowledge_base.pl	Facts, Rules and Knowledge base (Prolog)	Prolog File
templates/	HTML templates for the web interface	Directory
index.html	Landing page / Home	HTML File
dashboard.html	Real-time dashboard UI	HTML File
alerts.html	Alerts and recommendations page	HTML File
static/	Static files (CSS, JavaScript, Images)	Directory
css/	Stylesheets	Directory
style.css	Main stylesheet	CSS File
js/	JavaScript files	Directory
script.js	Frontend scripts for dashboard	JavaScript File
images/	Images, icons and other media	Directory
requirements.txt	List of Python dependencies	Text File
README.md	Project documentation and instructions	Markdown File
run.py	Script to run the application	Python File

LEGEND
Folder
Python File
Prolog File
HTML File
CSS File
JavaScript File
Text / Other File

NOTE
Computer Vision Module (cv_module/) is only used to observe and generate facts. All reasoning, decision making and output generation are handled by the Prolog Knowledge Base (knowledge_base.pl).

6. Modules of the System

The EduMind system is divided into multiple well-structured modules. Each module performs a specific task in the overall pipeline, starting from raw video input to final intelligent decision making through the Prolog Expert System. This modular design ensures scalability, maintainability, and clear separation between perception (Computer Vision) and reasoning (Expert System).

Module 1: Computer Vision Observation Module

Purpose:

This is the first and foundational module of the system. Its main purpose is to observe the student through webcam input and extract meaningful visual signals from the video frames.

Working:

Scope document for EduMind

- Captures live video stream using webcam
- Processes each frame using OpenCV
- Uses MediaPipe to detect face landmarks and hand landmarks
- Tracks key facial points such as eyes, nose, mouth, and head position
- Detects hand movement near face region

Output:

This module does NOT make any decisions. It only generates raw observations such as:

- eyes_closed
- looking_away
- head_down
- face_missing
- hand_near_face

These observations represent the system's "sensing layer", similar to human eyes.

Module 2: Feature Extraction & Observation Processing Module

Purpose:

This module converts raw visual landmark data into meaningful behavioral indicators that can be understood by the reasoning system.

Working:

- Calculates Eye Aspect Ratio (EAR) to detect eye closure
- Measures head orientation angles to detect head down or sideways posture
- Analyzes gaze direction based on eye position
- Detects proximity of hands to face region (possible distraction indicator)

Output:

The module converts numerical and geometric data into structured observations such as:

- student_is_distracted
- student_is_sleepy

Scope document for EduMind

- student_is_not_looking_at_screen

These outputs act as intermediate signals before being sent to the expert system.

Module 3: Fact Generation Module (Interface Layer)

Purpose:

This module acts as a bridge between Computer Vision and the Prolog Expert System.

Working:

- Receives observations from the CV module
- Converts them into Prolog-compatible facts
- Formats data into logical predicates
- Sends facts to the knowledge base for reasoning

Example Facts:

eyes_closed.

looking_away.

head_down.

Importance:

This module ensures that visual data is transformed into symbolic logic, which is essential for expert system processing.

Module 4: Prolog Expert System (Core Intelligence Module)

Purpose:

This is the MOST IMPORTANT module of the entire system. It acts as the decision-making brain of EduMind.

Working:

- Stores knowledge base of rules and facts
- Applies logical inference to interpret behavior
- Uses Backward Chaining to verify goals

Scope document for EduMind

- Uses Unification to match facts with rules
- Applies Rule-Based Reasoning to derive conclusions

Example Rule:

sleepy_student :-

eyes_closed,

head_down.

Output:

- attentive_student
- distracted_student
- sleepy_student
- disengaged_student
- phone_usage_suspected

This module provides explainable AI reasoning where every decision is based on logical rules.

Module 5: Backend Integration Module (Flask Server)

Purpose:

This module connects the frontend interface with the AI processing system.

Working:

- Handles API requests from frontend
- Sends observation data to backend processing pipeline
- Communicates with Prolog engine
- Returns final classification results

Features:

- REST API development using Flask
- Data routing between modules
- Session handling

Module 6: Frontend Dashboard Module

Purpose:

This module provides a user-friendly interface for real-time monitoring and visualization.

Working:

- Displays live student monitoring feed
- Shows attention level and behavior classification
- Displays recommendations and alerts
- Provides interactive UI controls

Features:

- Real-time updates using JavaScript
- Clean dashboard visualization
- Color-coded status indicators
- Result analytics panel

6.Working of Prolog Expert System (Core Intelligence)

The Prolog Expert System is the most important part of the EduMind project. It is responsible for all decision-making and reasoning. While Computer Vision only provides observations, Prolog interprets those observations and converts them into intelligent conclusions using logical rules.

Step 1: Receiving Facts from Computer Vision

The system receives raw observations generated by the Computer Vision module such as:

- eyes_closed
- looking_away
- head_down
- face_missing
- hand_near_face

These observations are passed as Prolog facts into the knowledge base.

Step 2: Storing Facts in Knowledge Base

All incoming observations are stored as logical facts inside the Prolog environment.

Example:

eyes_closed.

head_down.

looking_away.

These facts represent the current state of the student.

Step 3: Rule Matching Process

The Prolog engine starts comparing the received facts with predefined rules in the knowledge base.

Example rule:

sleepy_student :-

eyes_closed,

head_down.

This means: If eyes are closed AND head is down → student is sleepy.

Step 4: Unification Process

Prolog performs unification to match facts with rule conditions.

- It checks whether observed facts satisfy rule conditions
- It binds variables and matches patterns logically

This is a key mechanism that allows Prolog to reason intelligently.

Step 5: Backward Chaining Execution

The system uses backward chaining, which means:

- It starts from a goal (e.g., sleepy_student)
- Then checks whether required conditions (facts) are true

Scope document for EduMind

Example: Goal: Is student sleepy? → Check eyes_closed → Check head_down → If both true → goal is satisfied

Step 6: Rule-Based Inference

Once conditions are satisfied, Prolog applies rule-based reasoning to generate conclusions as :

- attentive_student
- distracted_student
- sleepy_student
- disengaged_student
- phone_usage_suspected

Step 7: Conflict Resolution (Multiple Rules)

If multiple rules are satisfied at the same time, the system evaluates all possible outcomes and selects the most relevant classification based on priority rules defined in the knowledge base.

Step 8: Generating Final Decision

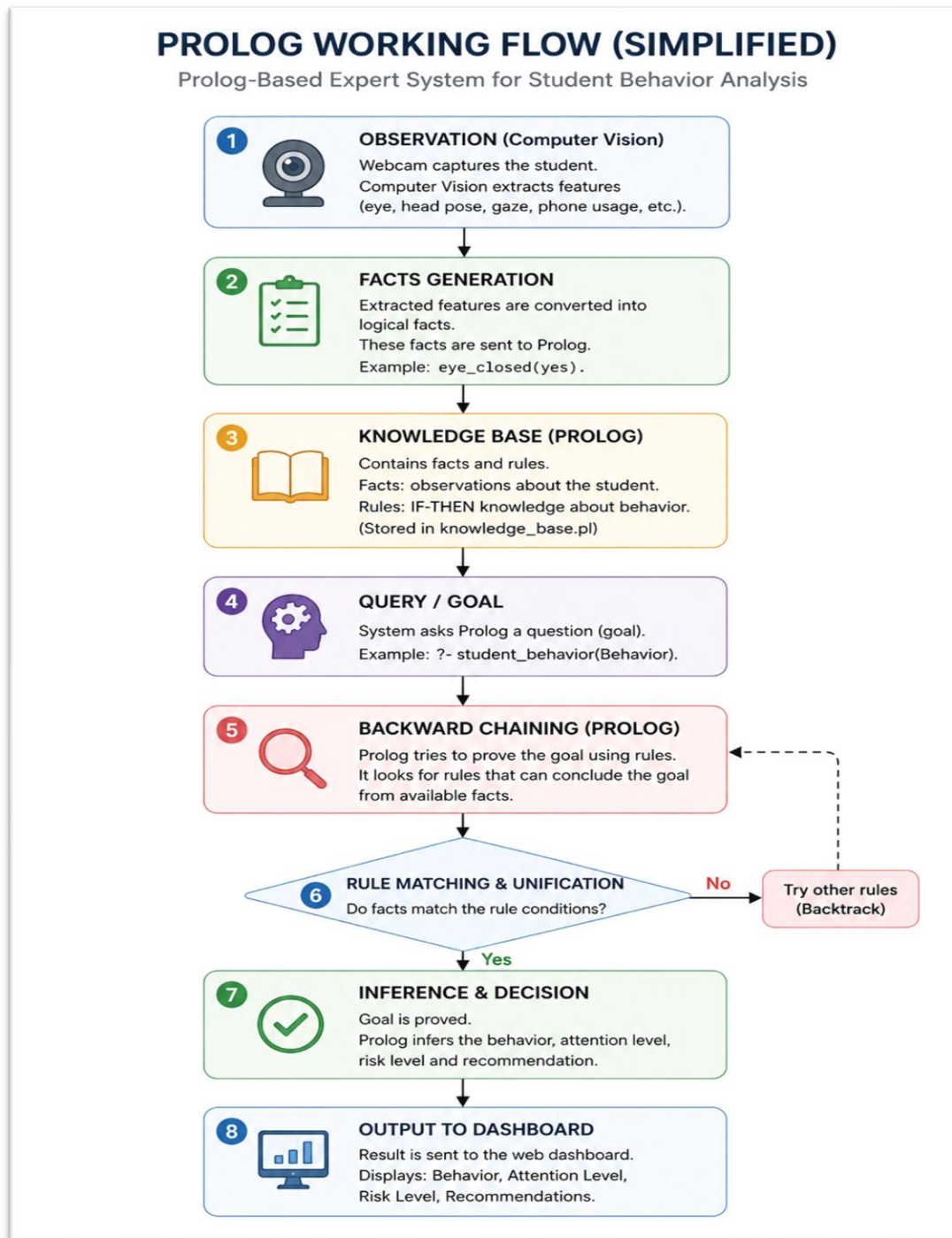
After successful reasoning, Prolog generates the final output:

- Student Behavior Classification
- Attention Level
- Risk Level
- Recommendation Signals

Step 9: Sending Results Back to System

The final decision is sent back to the Flask backend, which forwards it to the frontend dashboard for visualization.

Flowchart :



[Webcam]



[OpenCV / MediaPipe]



[Observation Generation]



[Prolog Facts]



[Expert System Reasoning]



[Behavior Classification]



[Dashboard Output]

7. System Limitations

- Requires webcam for operation
- Works for single user only
- Depends on lighting conditions
- Accuracy affected by occlusion
- Requires stable system performance

Future Enhancements

- Multi-student classroom support
- Deep learning integration
- Emotion recognition module
- Audio-visual fusion system
- Cloud-based deployment
- Mobile application version

8. Tools and Technologies

8.1 Tools and Technologies Used

Tool	Purpose
Python	Core programming language
OpenCV	Video processing
MediaPipe	Landmark detection
SWI-Prolog	Expert system engine
Flask	Backend framework
HTML/CSS/JS	Frontend development
NumPy	Numerical processing
VS Code	Development environment
Git	Version control

8.2 AI Techniques used :

- **Computer Vision (Observation Layer)**
- **Feature Extraction using MediaPipe**
- **Knowledge Representation (Prolog Facts & Rules)**
- **Rule-Based Reasoning**
- **Backward Chaining (Core inference method)**
- **Unification (Logical matching)**

9. GUI Mockups

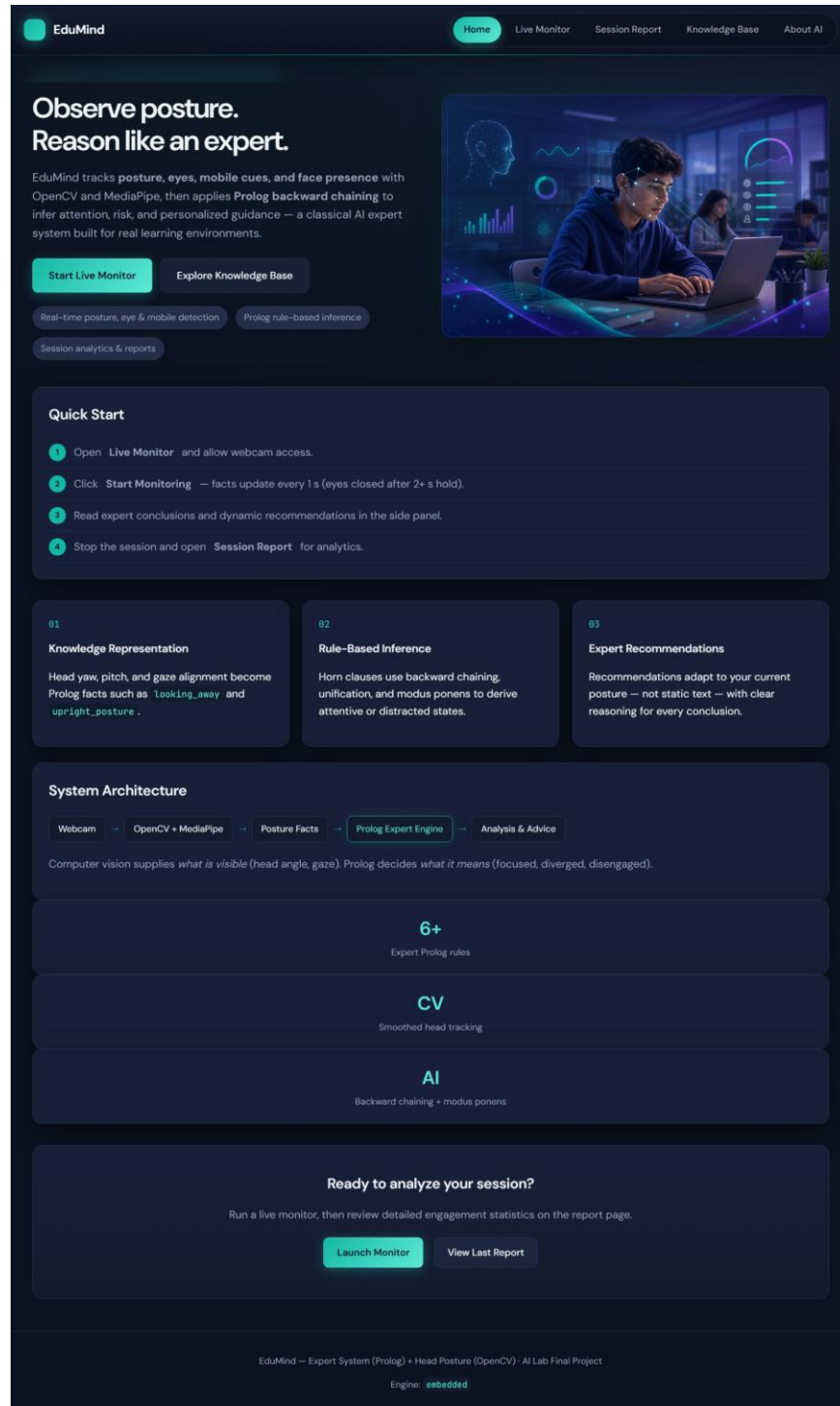


Figure 1 : Dashboard

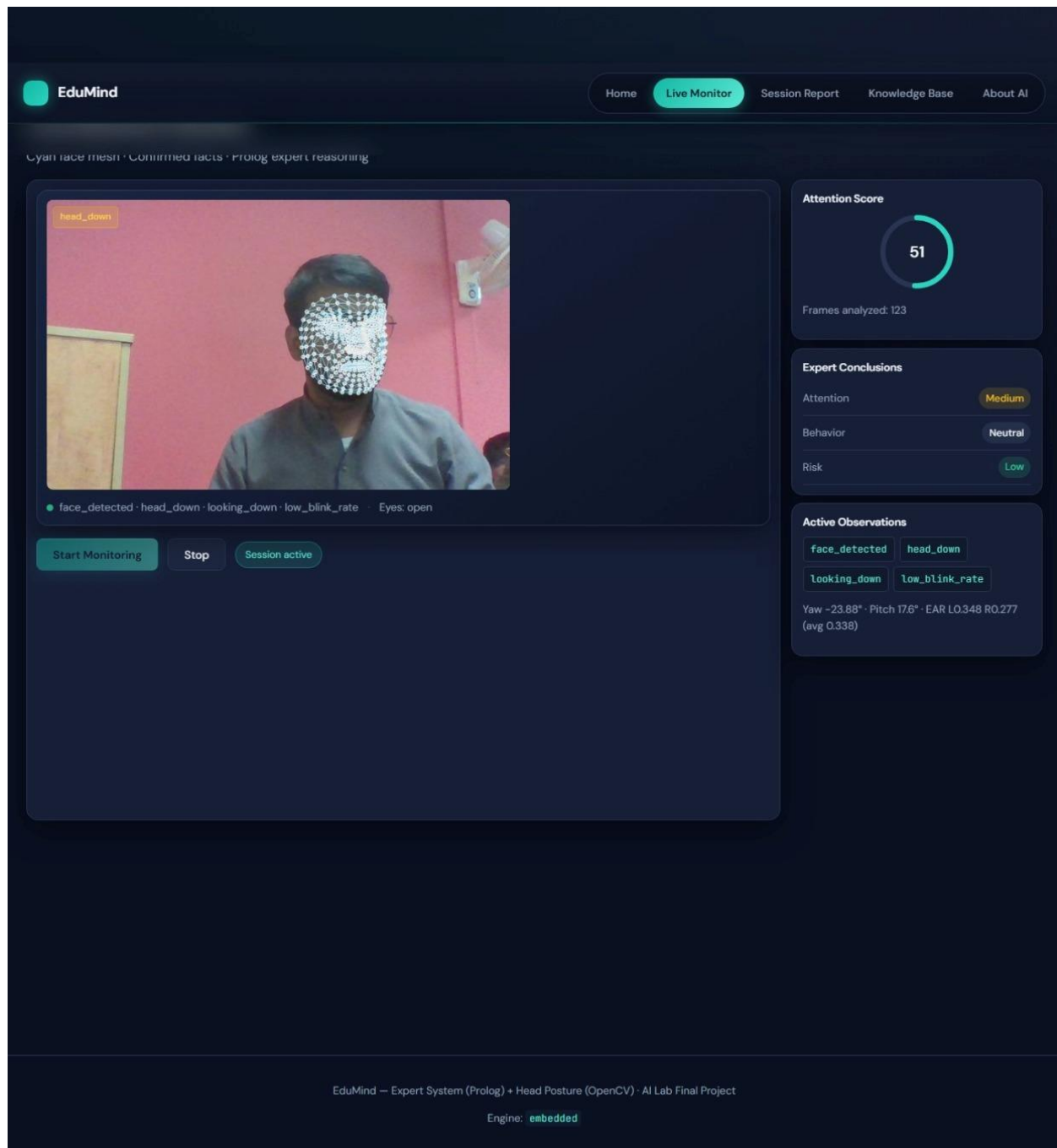


Figure 2 : Live Monitoring Board

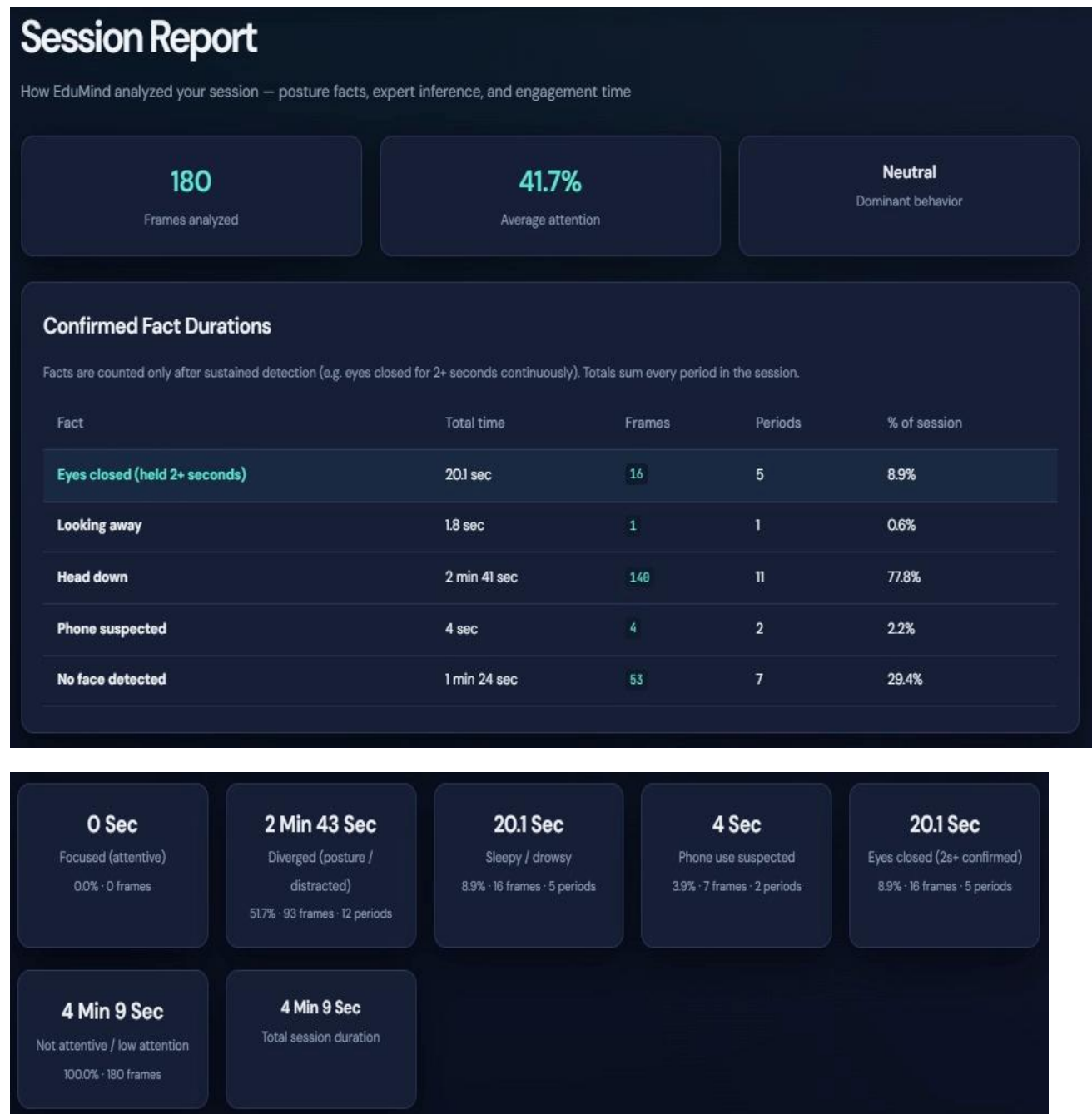


Figure 3 : Session Report



Figure 4 : Attention Timeline Graph

Expert System Applications

Points where backward chaining, unification, modus ponens, and inference produced results:

Time (s)	Technique	Rule / Step	Result
0.0	Unification	assert_observation/1	Facts bound: no_face.
0.0	Backward chaining	disengaged_student	No face visible.
9.0	Unification	assert_observation/1	Facts bound: eye_contact, face_detected, low_blink_rate.
9.0	Unification	partially_attentive	Mixed signals — monitoring continues.
10.0	Unification	assert_observation/1	Facts bound: none.
13.0	Unification	assert_observation/1	Facts bound: eye_contact, face_detected, looking_down, low_blink_rate.
18.0	Unification	assert_observation/1	Facts bound: face_detected.


EduMind

[Home](#)
[Live Monitor](#)
[Session Report](#)
[Knowledge Base](#)
[About AI](#)

[< BACK](#)

Prolog Knowledge Base

Posture, eyes, mobile, and face facts — Horn clauses with backward-chaining inference

Dynamic Facts

Observations from the posture monitor are asserted at runtime:

```

observation(eyes_closed).
observation(looking_away).
observation(head_down).
observation(phone_suspected).
observation(hand_near_face).
observation(no_face).
observation(face_detected).

```

Inference Techniques

- **Backward chaining** — prove goals like `attentive_student` by matching rule heads
- **Unification** — variables bind during SLD resolution across clauses
- **Modus ponens** — `attention_level(high) :- attentive_student`
- **Inference** — derive risk and behavior from proven posture rules
- **Expert system** — separates knowledge, inference engine, and explanation

Expert Rules Catalog

R1
sleepy_student

backward chaining

```

sleepy_student :- eyes_closed,
head_down.

```

Infers drowsiness when eyes are closed and head is lowered.

R2
distracted_student

backward chaining

```

distracted_student :- looking_away,
phone_suspected.

```

Infers distraction from gaze away plus suspected phone use.

R3
attentive_student

backward chaining

```

attentive_student :- eye_contact,
upright_posture, \+ eyes_closed, \+
phone_suspected.

```

Infers high engagement when posture, gaze, and eyes align.

R4
attention_level(high)

modus ponens

```

attention_level(high) :-
attentive_student.

```

Maps attentive behavior to high attention level.

R5
phone_usage_suspected

backward chaining

```

phone_usage_suspected :- hand_near_face,
looking_down.

```

Heuristic for mobile device use from hand and gaze cues.

R6
disengaged_student

expert system

```

disengaged_student :- no_face, \+
face_detected.

```

Flags disengagement when no face is visible.

Sample Horn Clauses

```

sleepy_student :-
  observation(eyes_closed),
  observation(head_down).

phone_usage_suspected :-
  observation(hand_near_face),
  observation(looking_down).

attentive_student :-
  observation(eye_contact),
  observation(upright_posture),
  \+ observation(eyes_closed),
  \+ observation(phone_suspected).

attention_level(high) :- attentive_student.

```

Reasoning Flow

- CV asserts posture facts
- Backward chaining proves behavior
- Modus ponens maps to attention level
- Expert system outputs recommendation

Try Live Monitor

EduMind — Expert System (Prolog) + Head Posture (OpenCV) - AI Lab Final Project

Engine: `embedded`

Figure 5 : Prolog Knowledge Base

EduMind Home Live Monitor Session Report Knowledge Base **About AI**

[← BACK](#)

About EduMind & AI Concepts

Problem statement, scope, and viva-ready explanations

Problem Statement

Students in online or hybrid classrooms often disengage silently — looking away, using phones, or showing fatigue. Teachers cannot monitor every learner continuously. EduMind acts as an intelligent classroom observer that converts visual cues into logical facts and applies expert rules to infer behavior and recommend corrective actions.

Project Scope

- Single-user laptop webcam monitoring
- Behavior fact extraction (not raw CV decisions)
- Prolog-based expert reasoning engine
- Session analytics and recommendations
- Educational GUI for demonstration and viva

AI Course Mapping

Concept	Implementation in EduMind
Knowledge representation	observation/1 facts from CV module
Rules	Horn clauses in prolog/knowledge_base.pl
Inference	Prolog backward chaining (SLD resolution)
Expert system	Separate observation, KB, inference, explanation layers
Reasoning	Logical derivation of attention, risk, advice
Computer vision	OpenCV + MediaPipe — observations only

Why OpenCV Does Not “Decide”

The CV layer answers: *what is visible?* (eyes closed, gaze away, hands near face). The Prolog layer answers: *what does it mean?* (sleepy, distracted, attentive). This separation is the hallmark of a classical expert system and distinguishes this project from a pure computer-vision application.

EduMind — Expert System (Prolog) + Head Posture (OpenCV) · AI Lab Final Project

Engine: [embedded](#)

Figure 6 : About of the Project

10. Deployment of the System

The EduMind web application has been successfully deployed online using **Render.com** cloud hosting platform. The deployment allows the system to run as a live web-based application that can be accessed through a browser without requiring local setup on every machine.

The Flask backend, frontend dashboard, and AI processing modules are integrated and hosted on Render. This deployment demonstrates the practical implementation of the project in a real-world environment and shows that the system is not only a prototype but also a functional web application.

Deployment Platform:

- Render.com

Deployment Features:

- Online accessibility
- Flask web application hosting
- Real-time dashboard support
- Backend and frontend integration
- Cloud-based execution

 Link : <https://edumind-t0h9.onrender.com>

11. Conclusion

EduMind successfully demonstrates a hybrid Artificial Intelligence system where Computer Vision acts as a perception layer and Prolog-based Expert System acts as the intelligence core. The system effectively analyzes student behavior by converting visual observations into logical facts and applying rule-based reasoning.

This separation of perception and reasoning makes the system highly explainable, modular, and aligned with classical AI principles. The project highlights the power of Expert Systems in real-world educational applications.

12. References

1. Artificial Intelligence: A Modern Approach – Russell & Norvig
2. AI Lab Manual – CSC262
3. SWI-Prolog Documentation

http://swi-prolog.org/pldoc/doc_for?object=root

4. OpenCV Documentation

<https://docs.opencv.app/en/>

5. Flask Documentation

<https://flask.palletsprojects.com/en/stable/>

6. Python Official Documentation

<https://docs.python.org/3/tutorial/index.html>